

Mini-project 1 in Advanced Machine Learning (02460)

by Morten (s224022), Holger (s214776) & Karol (s243920) on 05 March 2026

Variational Autoencoders' priors

The standard VAE uses Gaussian prior. Through ELBO, the aggregated posterior structure of latent distribution (KL distance wise) is pushed to resemble the prior. However, latent space distribution often is not Gaussian. That may lead to mismatch, and empty 'holes' where sampling quality is low. That is aimed to be addressed by learnable priors, mainly Mixture of Gaussians (MoG), and Flow based priors.

At the plot 1, we indeed see that MoG and Flow based priors offer more flexibility for the encoder's latent space distribution shape. That, as can be observed, often results in more distinct clusters.

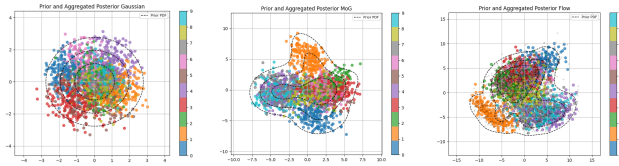


Figure 1: Prior (contour) and aggregated posterior (samples) for three prior choices: Gaussian, Mixture of Gaussian, Flow based. Projection onto 2D plane using PCA, and Monte Carlo sampling for flow based prior.

Test set ELBO loss		
Prior	Mean	Variance
Gaussian	86.282827	0.034448
MoG	85.226179	0.088758
Flow	84.054018	0.149392

Table 1: Summary statistics of test set loss (negative ELBO approximated) across priors (10 runs per prior).

Looking at table 1 we can see that indeed learnable priors bring us to a lower test set loss. Yet, it comes on expense of higher variance across tests.

The encoder network ($q_\phi(z|x)$) maps an input image $x \in \mathbb{R}^{28 \times 28}$ to the parameters of a Gaussian latent distribution. It consists of two fully connected layers with ReLU activations ($784 \rightarrow 512 \rightarrow 512$). The final linear layer outputs 2M values ($\mu(x)$ and log-standard-deviation vectors).

The decoder network ($p_\theta(x|z)$) maps back to the image space. It consists of two fully connected layers with ReLU activations ($M \rightarrow 512 \rightarrow 512$). The final linear layer outputs 784 (28×28) values.

Deep Diffusion Probabilistic Models

The standard DDPM presented in Ho et al. (2020) unlike standard VAE's does not operate in latent space but directly in the image space. The model is trained by continuously adding gaussian noise to an input image until it becomes pure noise. By learning to re-

verse the noising process, the model is able to generate images from pure noise. However, sampling from a DDPM is computationally expensive due to the large number of sequential denoising steps. To address this, we explore a Latent DDPM that operates in the latent space of a β -VAE. Figure 2 shows samples from a VAE with flow prior, a DDPM, and a Latent DDPM trained on MNIST.

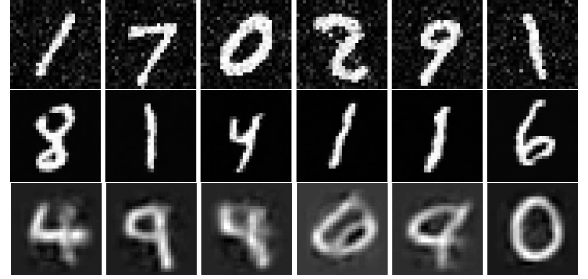


Figure 2: Six samples from each: VAE (flow prior), DDPM and Latent DDPM respectively

Qualitatively the DDPM generates the better samples while the Latent DDPM is worst. However evaluated on the Fréchet Inception Distance Latent DDPM performs better than VAE, table 2. The table also highlights that sampling from DDPM is slower by a factor 10^4 . On figure 3 we observe that the VAE prior is insufficient to match the true latent distribution of the data. However the learned latent DDPM follows the aggregate posterior indicating the latent DDPM learns a flexible prior that adapts to the true latent distribution.

	VAE	DDPM	Latent DDPM		
			$\beta = 10^{-6}$	$\beta = 10^{-4}$	$\beta = 10^{-2}$
FID	6.98	6.07	6.60	7.07	9.27
s/batch	0.018	134.78	0.15	0.15	0.16

Table 2: FID scores and sampling times for batches of size 256 for DDPM, VAE (Flow prior), and Latent DDPM with different β values.

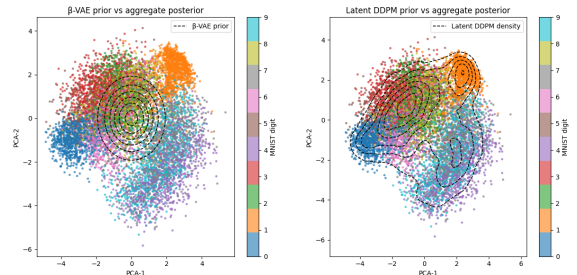


Figure 3: β -VAE prior and Learned Latent DDPM distribution against aggregated posterior projected to 2D space by PCA.

Code snippets

The following code snippet implements Algorithm 1 of [Ho et al. \(2020\)](#)

```

1 def negative_elbo(self, x):
2     ### Implement Algorithm 1 here ###
3
4     t = torch.randint(low=1, high=self.T, size=(x.shape[0],), device=x.device)
5     t = t[:, None]
6     epsilon = torch.randn_like(x)
7     alpha_bar_t = self.alpha_cumprod[t]
8     alpha_bar_t = alpha_bar_t.view(x.shape[0], 1)
9
10    x_t = torch.sqrt(alpha_bar_t)*x + torch.sqrt(1- alpha_bar_t)*epsilon
11    neg_elbo = (epsilon - self.network(x_t, t))*2
12
13    return neg_elbo

```

The following code snippet shows the construction of the Flow prior using masked coupling layers.

```

1 mask = torch.tensor([1 if i % 2 == 0 else 0 for i in range(M)], dtype=torch.float32)
2
3 for _ in range(num_transformations):
4     mask = 1 - mask
5     scale_net = nn.Sequential(
6         nn.Linear(M, hidden_dim),
7         nn.ReLU(),
8         nn.Linear(hidden_dim, M),
9         nn.Tanh(),
10    )
11    translation_net = nn.Sequential(
12        nn.Linear(M, hidden_dim),
13        nn.ReLU(),
14        nn.Linear(hidden_dim, M),
15    )
16    transformations.append(MaskedCouplingLayer(scale_net, translation_net, mask))
17
18 self.flow = Flow(base, transformations)

```

The following code snippet implements Algorithm 2 of [Ho et al. \(2020\)](#)

```

1 def sample(self, shape):
2     # Sample x_t for t=T (i.e., Gaussian noise)
3     x_t = torch.randn(shape).to(self.alpha.device)
4
5     # Sample x_t given x_{t+1} until x_0 is sampled
6     for t in range(self.T-1, -1, -1):
7         ### Implement the remaining of Algorithm 2 here ###
8         alpha_t = self.alpha[t]
9         sqrt_alpha_t = torch.sqrt(self.alpha[t])
10        sqrt_beta_t = torch.sqrt(self.beta[t])
11        sqrt_one_minus_alpha_bar_t = torch.sqrt(1-self.alpha_cumprod[t])
12
13        if t > 1:
14            z = torch.randn_like(x_t)
15        else: z = torch.zeros_like(x_t)
16
17        x_t = 1/sqrt_alpha_t * (x_t - (1-alpha_t)/(sqrt_one_minus_alpha_bar_t)*self.
18            network(x_t, torch.full((shape[0],1), t, device = self.alpha.device))) + sqrt_beta_t*z
19
20    return x_t

```

The Following code snippet shows the implementation of the the β -VAE ELBO

```

1 def elbo(self, x):
2
3
4     q = self.encoder(x)
5     z = q.rsample()
6
7     elbo = torch.mean(self.decoder(z).log_prob(x) - self.beta * td.kl_divergence(q, self.prior()), dim=0)
8
9     return elbo

```

The following code snippet shows how we train the Latent DDPM, which we do in a two step manner. First we train the β -VAE and "freezes" its parameters. Then we use samples from the β -VAE encoder to train a DDPM on the latent space.

```

1 def train(vae, ddpm, optimizer_vae, optimizer_ddpm, data_loader,
2           vae_epochs, ddpm_epochs, device):
3
4     #Train beta-VAE
5     vae.train()
6     for epoch in range(vae_epochs):
7         for batch in tqdm(data_loader, desc=f"VAE Epoch {epoch+1}/{vae_epochs}"):
8             x = batch[0].to(device) # flattened input
9             optimizer_vae.zero_grad()
10            loss = vae(x)
11            loss.backward()
12            optimizer_vae.step()
13
14        # Freeze VAE for DDPM
15        vae.eval()
16        for p in vae.parameters():
17            p.requires_grad = False
18
19        #Train latent DDPM on latent space
20        ddpm.train()
21        for epoch in range(ddpm_epochs):
22            for batch in tqdm(data_loader, desc=f"DDPM Epoch {epoch+1}/{ddpm_epochs}"):
23                x = batch[0].to(device)
24
25                # Sample latent z from VAE encoder
26                with torch.no_grad():
27                    q = vae.encoder(x)
28                    z0 = q.rsample() # [batch, latent_dim]
29
30                optimizer_ddpm.zero_grad()
31                loss = ddpm.loss(z0)
32                loss.backward()
33                optimizer_ddpm.step()

```

The following code snippet shows how we sample from the Latent DDPM. First we sample from the DDPM, these sample are then passed through the β -VAE decoder.

```

1 ddpm.eval()
2 vae.eval()
3
4 with torch.no_grad():
5     z_samples = (ddpm.sample((args.batch_size, 10))).to(args.device)
6     samples = vae.decoder(z_samples)

```

The architecture of DDPM is as follows.

```

1 # DDPM network
2 network = FcNetwork(M, 256)
3
4 # Encoder network
5 encoder_net = nn.Sequential(
6     nn.Flatten(),
7     nn.Linear(784, 512),
8     nn.ReLU(),
9     nn.Linear(512, 512),
10    nn.ReLU(),
11    nn.Linear(512, M*2),
12 )
13
14 # Decoder network
15 decoder_net = nn.Sequential(
16     nn.Linear(M, 512),
17     nn.ReLU(),
18     nn.Linear(512, 512),
19     nn.ReLU(),
20     nn.Linear(512, 784),
21 )
22

```

Declaration of use of generative AI

This declaration **must** be filled out and included as the 3rd page of the document. The questions apply to all parts of the work, including research, project writing, and coding.

- I/we have used generative AI tools: [yes]

If you answered *yes*, please complete the following sections. List the generative AI tools you have used:

- ChatGPT

Describe how the tools were used:

What did you use the tool(s) for? We have used ChatGPT for debugging and rephrasing.

At what stage(s) of the process did you use the tool(s)? In the coding stage for GPU training (finding missing `to.device()`) and for cleaning up our visualizations. We also used it in the reporting to rephrase our sentences to make them shorter and more precise

How did you use or incorporate the generated output? Read through the suggested code and implemented the parts that seemed reasonable. For reporting we had it rephrase are sentences and used the parts that were reasonable.

References

- J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *arXiv preprint arXiv:2006.11239*, 2020.
- J. M. Tomczak. *Deep Generative Modeling*. Springer, 2 edition, 2024. Second Edition.